

MINI GIT

구현 기반 개념서

커밋 그래프 · BFS/DFS · 위상 정렬 · 역색인 · 안정 병합 정렬

이 문서는 B4-2의 실제 main.py 구조와 동작을 기준으로, 요구사항 PDF의 필수 개념과 첨부 평가 질문 20개에 바로 답할 수 있도록 구성했습니다. 선택 과제인 Diff, Merge, 성능 비교는 구현 범위에서 제외했습니다.

B4-2 | Python 3.10+ | 메모리 기반 CLI

01 / 07

무엇을 만들었는가

한 문장 설명

파일 내용 자체가 아니라 커밋 메타데이터의 변경 이력을 메모리에 기록하고, 브랜치와 그래프 탐색을 제공하는 작은 Git 학습용 REPL입니다.

명령별 동작

명령	구현 동작	핵심 확인점
INIT user	상태 초기화, main 생성, HEAD와 사용자 설정	HEAD = current_branch
BRANCH name	현재 브랜치의 커밋 hash를 복사	분기점 보존
SWITCH name	현재 브랜치만 변경	이후 COMMIT 대상 변경
COMMIT message	부모 연결, hash 생성, 브랜치 이동, 인덱스 갱신	원자적 생성 흐름
LOG	Kahn 방식으로 부모 우선 출력	최신순이 아님
PATH a b	부모 연결을 무방향으로 보고 BFS	최단, 동률은 사전순
ANCESTORS h	부모 방향으로 모든 조상 탐색	자기 자신 제외
SEARCH keyword	keyword 역색인 후보 조회	split + lower
SEARCH --author=name	author 역색인 후보 조회	casefold 정규화
LOG --sort-by=...	직접 구현한 안정 병합 정렬	date 또는 author

범위 경계

파일 내용 추적, 네트워크, 파일 영속성, Diff, Merge는 구현하지 않았습니다. 요구사항의 필수 기능만 담았습니다.

02 / 07

상태와 자료구조

구성 요소	형태	책임
Commit	불변 dataclass	hash, message, author, timestamp, parents 보관
commits	dict(hash, Commit)	hash 평균 O(1) 조회와 유일성 검사
branches	dict(name, hash None)	브랜치별 끝 커밋을 가리킴
current_branch	str	현재 HEAD 역할
keyword_index	dict(token, list(hash))	메시지 토큰 검색 후보 제공
author_index	dict(author, list(hash))	작성자 검색 후보 제공
_commit_order	dict(hash, int)	검색 결과 생성 순서 복원

부모 -> 자식 방향의 DAG, 부모 hash만 노드에 저장



커밋 생성 순서

단계	main.py에서 일어나는 일
1	현재 브랜치가 가리키는 hash를 parents로 결정
2	증가 카운터로 세션 내 유일한 hash 생성
3	Commit을 commits 해시맵에 삽입
4	현재 브랜치 포인터를 새 hash로 이동
5	keyword와 author 역색인을 즉시 갱신

왜 DAG인가

새 커밋은 이미 존재하는 현재 HEAD만 부모로 가리킵니다. 시간상 나중 노드에서 이전 노드로만 연결되므로 사이클이 생기지 않습니다. 사이클이 있으면 조상 의미가 깨지고 탐색이 반복되며 위상 순서가 존재하지 않습니다.

03 / 07

그래프 탐색 세 가지

기능	알고리즘	핵심 구현	복잡도
LOG	Kahn 위상 정렬	부모 수가 0인 노드부터 큐에 넣고 자식의 남은 부모 수를 감소	$O(V+E)$
PATH	BFS	부모-자식 양방향 인접 목록, 이웃 hash를 직접 정렬, predecessor로 복원	$O(V+E)$, 이웃 정렬 별도
ANCESTORS	DFS 계열	stack으로 부모만 따라가며 set으로 중복 방문 방지	$O(V+E)$

LOG: 부모가 먼저 나오는 이유

- 각 노드의 남은 부모 수를 계산합니다. 0인 커밋만 출력 대기 큐에 들어갑니다.
- 한 커밋을 출력한 뒤에야 그 자식의 남은 부모 수를 줄입니다. 모든 부모가 출력된 자식만 큐에 들어가므로 부모 우선 조건이 보장됩니다.
- 출력 개수가 전체 커밋 수보다 작으면 사이클이 있다는 뜻이므로 방어적으로 오류를 냅니다.

PATH: 최단 경로와 사전순 동물 처리

- 요구사항에 따라 부모 연결을 무방향으로 취급해 부모와 자식 양쪽으로 이동할 수 있습니다.
- BFS는 간선 수가 같은 경로를 총별로 탐색하므로 처음 찾은 경로가 최단입니다.
- 각 노드의 이웃 hash를 직접 구현한 병합 정렬로 사전순 방문합니다. 따라서 같은 길이의 후보 중 경로 문자열이 가장 작은 것이 먼저 발견됩니다.
- 큐에는 전체 경로 대신 노드만 넣고 predecessor를 기록한 뒤 목적지에서 역추적하여 메모리 낭비를 줄였습니다.

ANCESTORS: 빠짐없는 조상

- 시작 커밋의 parents를 stack에 넣고 부모 방향만 반복합니다. found set이 중복과 재방문을 막습니다.
- 발견 집합을 부모 우선 LOG 순서로 필터링해 오래된 조상이 먼저 보이도록 출력합니다.

04 / 07

역색인과 직접 구현한 정렬

역색인: 커밋을 모두 읽지 않고 후보로 바로 이동

COMMIT 시점 갱신

메시지를 `split()`하고 `lower()`한 토큰을 `keyword_index(token)`에 추가합니다. 작성자는 `casefold()`한 값을 `author_index(author)`에 추가합니다. 같은 메시지에 같은 토큰이 반복되어도 `set`으로 한 번만 기록합니다.

질의	순회 검색	역색인
keyword	모든 N개 메시지를 검사: $O(N \times L)$	토큰 hash 조회 평균 $O(1)$ + 결과 K개 처리
author	모든 N개 작성자를 검사: $O(N)$	작성자 hash 조회 평균 $O(1)$ + 결과 K개 처리

안정 병합 정렬

- `merge_sort`는 입력을 반으로 나누고 각 절반을 재귀 정렬한 뒤 병합합니다. 표준 `sorted()`와 `list.sort()`는 사용하지 않습니다.
- 동일 키에서는 왼쪽 원소를 먼저 선택합니다. 그래서 같은 날짜나 작성자의 기존 상대 순서가 유지되는 안정 정렬입니다.
- 평균과 최악 모두 $O(N \log N)$, 추가 메모리 $O(N)$ 입니다. 퀵 정렬보다 최악 성능이 예측 가능하고 안정성이 필요한 출력에 적합합니다.

연산	평균	최악	메모
hash 조회	$O(1)$	$O(N)$	총돌이 극단적인 이론적 최악
LOG	$O(V+E)$	$O(V+E)$	DAG 위상 출력
PATH BFS	$O(V+E)$	$O(V+E)$	인접 목록 구성, 이웃 정렬 비용 제외
ANCESTORS	$O(V+E)$	$O(V+E)$	도달 부분 + 출력 순서 계산
병합 정렬	$O(N \log N)$	$O(N \log N)$	안정, $O(N)$ 보조 메모리
역색인 검색	$O(1)+O(K)$	$O(K \log K)$	후보의 생성 순서 정렬 포함

05 / 07

질문지 답변: 기능과 설계

항목 1 - 동작 확인

Q1-1. INIT 후 main, HEAD, 사용자가 초기화되는가?

A. 예. initialize()가 기존 메모리 상태를 비우고 branches={main: None}, current_branch=main, current_user=입력값으로 설정합니다.

Q1-2. BRANCH, SWITCH 뒤 COMMIT이 선택 브랜치에 반영되는가?

A. 예. BRANCH는 현재 hash를 복사하고, SWITCH는 current_branch를 바꿉니다. COMMIT은 branches(current_branch)를 새 hash로 갱신합니다.

Q1-3. LOG에서 부모가 자식보다 먼저 나오는가?

A. 예. topological_commits()가 Kahn 위상 정렬을 사용해 모든 부모가 처리된 뒤에만 자식을 출력합니다.

Q1-4. PATH가 최단 경로 또는 No path를 출력하는가?

A. 예. 무방향 BFS로 최단 경로를 찾고, 연결되지 않았으면 None을 반환해 CLI가 No path를 출력합니다. 동물은 사전순 이웃 방문으로 해결합니다.

Q1-5. ANCESTORS가 모든 조상을 출력하는가?

A. 예. stack과 found set으로 부모 방향의 모든 도달 노드를 한 번씩 방문하며 시작 커밋 자신은 제외합니다.

Q1-6. SEARCH와 정렬 옵션이 동작하는가?

A. 예. keyword/author 역색인으로 후보를 가져오고, LOG --sort-by=date|author는 직접 구현한 안정 병합 정렬을 사용합니다.

항목 2 - 구조와 책임

Q2-1. 커밋 저장소, 브랜치, HEAD, 사용자 정보는 어떻게 분리했는가?

A. Commit은 노드 데이터, commits는 hash 조회, branches는 이름별 끝점, current_branch는 HEAD, current_user는 새 커밋 작성자를 각각 담당합니다.

Q2-2. hash 빠른 조회와 중복 방지는?

A. dict의 hash -> Commit 구조로 평균 O(1) 조회합니다. 증가 카운터 후보를 만들고 commits에 없는지 확인하므로 세션 내 중복이 없습니다.

Q2-3. 역색인은 언제 어떻게 갱신되는가?

A. 커밋을 저장하고 브랜치 포인터를 이동한 직후 _index_commit()을 호출합니다. 메시지 토큰과 작성자에 해당 hash를 추가합니다.

Q2-4. LOG, PATH, ANCESTORS의 탐색 로직 재사용 구조는?

A. 모두 MiniGitRepository의 독립 메서드이며 공통 commits/parents 모델을 읽습니다. 위상 출력은 LOG와 ANCESTORS 출력 순서에 재사용됩니다.

Q2-5. docstring과 주석 기준은?

A. 공개 클래스, 주요 함수, 알고리즘 메서드에는 책임과 보장을 docstring으로 적었습니다. 안정성, hash 유일성처럼 코드만으로 의미가 불명확한 지점만 인라인 주석을 썼습니다.

06 / 07

질문지 답변: 알고리즘 이해

Q3-1. 커밋 그래프가 왜 DAG여야 하며 사이클이 생기면 무엇이 문제인가?

A. 커밋은 과거 상태를 부모로 가리키는 이력이라 방향성이 있고 되돌아 자신에게 도달하면 안 됩니다. 사이클이 있으면 조상/후손 의미가 모순되고 단순 DFS가 무한 반복할 수 있으며 위상 정렬이 불가능합니다.

Q3-2. LOG의 부모 우선 조건을 어떻게 만족시켰는가?

A. 각 커밋의 부모 수를 진입 차수처럼 두고 0인 노드부터 출력합니다. 부모가 출력될 때 자식의 수를 감소시켜 0이 된 자식만 큐에 넣는 Kahn 방식입니다.

Q3-3. PATH에 왜 BFS를 쓰고 간선을 왜 무방향으로 보는가?

A. 모든 간선 비용이 1이므로 BFS의 첫 도달이 간선 수 최소 경로입니다. 요구사항이 커밋-부모 연결의 양쪽 이동을 허용하므로 child -> parent와 parent -> child를 모두 인접 목록에 넣습니다.

Q3-4. 정렬의 평균/최악 시간복잡도와 안정성은?

A. 병합 정렬은 분할 단계가 $\log N$ 층이고 각 층 병합이 $O(N)$ 이므로 평균과 최악 모두 $O(N \log N)$ 입니다. 같은 키에서 왼쪽을 먼저 뽑아 기존 상대 순서를 보존하므로 안정 정렬입니다.

Q3-5. 역색인이 순회 검색보다 빠른 이유는?

A. 순회는 매 검색마다 N개 커밋과 메시지를 검사합니다. 역색인은 COMMIT 시 미리 token/author -> hash 목록을 만들기 때문에 검색은 dict 조회 뒤 실제 결과 K개만 다룹니다.

말로 설명할
때의 순서

정의 -> 내 코드의 자료구조 -> 알고리즘 선택 이유 -> 시간복잡도 -> 경계 사례 순으로
답하면 구현과 개념이 자연스럽게 연결됩니다.

경계 사례 세 가지

- 초기 커밋은 parents=()이며, 첫 커밋 전에 나눈 두 브랜치에서 각각 커밋하면 서로 단절되어 PATH는 No path가 될 수 있습니다.
- PATH의 시작과 끝 hash가 같으면 길이 0인 경로 {hash}를 반환합니다.
- 검색 결과가 없거나 로그가 비어 있으면 No commits를 출력하고, 모르는 hash는 Unknown commit: hash로 구분합니다.

07 / 07

질문지 답변: 변경과 확장 사고

Q4-1. 커밋 수가 10배가 되면 병목과 개선 방향은?

A. 정렬은 $O(N \log N)$, LOG와 전체 그래프 탐색은 $O(V+E)$ 로 커집니다. 현재 PATH는 호출마다 무방향 인접 목록과 이웃 정렬을 다시 만듭니다. children/adjacency를 커밋 시 함께 갱신해 캐시하고, 반복 LOG는 변경 버전별 결과를 캐시할 수 있습니다. 인덱스는 이미 전체 순회를 피합니다.

Q4-2. PATH를 부모 방향만 허용하면 무엇이 달라지는가?

A. 자식에서 조상으로만 이동할 수 있어 ancestor $\rightarrow$ descendant 경로는 No path가 됩니다. shortest_path()에서 parent_hash $\rightarrow$ child_hash 역방향 추가를 제거하고 commit.parents만 이웃으로 사용하면 됩니다. BFS 자체는 유지됩니다.

Q4-3. author 정렬과 부모 우선 조건을 동시에 지키려면?

A. 전체 목록을 author로 다시 정렬하면 부모가 뒤로 밀릴 수 있습니다. Kahn 알고리즘의 ready 집합, 즉 현재 출력 가능한 커밋들 안에서만 author를 우선순위로 선택해야 합니다. 동렬은 기존 순서나 date/hash 규칙으로 안정 처리합니다.

Q4-4. hash를 카운터에서 난수로 바꾸면 어떤 영향이 있는가?

A. 충돌 검사를 반복해야 하고 테스트에서 hash를 고정하려면 난수 생성기를 주입하거나 seed/mock이 필요합니다. 실행마다 값이 달라 재현 로그 비교와 디버깅이 어려워집니다. 충분한 비트 수와 충돌 재시도는 필수입니다.

30초 요약

데이터	Commit 노드는 불변, commits는 hash 조회, branches는 끝점, current_branch가 HEAD, 두 역색인은 검색 후보를 제공합니다.
그래프	커밋은 과거 부모만 가리켜 DAG입니다. LOG는 Kahn, PATH는 무방향 BFS, ANCESTORS는 부모 방향 DFS 계열입니다.
정렬/검색	직접 구현한 안정 병합 정렬은 $O(N \log N)$ 이고, 역색인은 검색 시 전체 커밋 순회를 피합니다.

검증 근거

- test_main.py의 9개 테스트가 초기화, 분기, HEAD 반영, 부모 우선 LOG, 단절 경로, 조상, 인덱스, 안정 정렬, CLI와 REPL을 검증합니다.
- 실행: `python3 main.py` | 테스트: `python3 -m unittest -v`

끝. 이 문서의 답변은 B4-2 main.py의 현재 구현을 기준으로 합니다.